

บทที่ 5

โพรโทคอล HTTP (HyperText Transfer Protocol)

HTTP เป็นโพรโทคอลที่ใช้ในการติดต่อสื่อสารระหว่างเว็บเบราว์เซอร์และเว็บเซิร์ฟเวอร์ สำหรับเครือข่ายเวิลด์ไวด์เว็บ โดยใช้คอนเนกชันของไคลเอนต์-เซิร์ฟเวอร์เป็นหลัก หลักการทำงานของ HTTP คือการส่งข้อความจากไคลเอนต์ไปยังเซิร์ฟเวอร์เพื่อตีความหมาย หลังจากนั้นเซิร์ฟเวอร์จะทำการส่งข้อความซึ่งเป็นผลการตีความกลับมายังไคลเอนต์ โดยทั่วไปการส่งข้อความระหว่างไคลเอนต์และเซิร์ฟเวอร์มักใช้ไบต์สตรีมเป็นหลัก แต่ HTTP เป็นโพรโทคอลที่ออกแบบมาเพื่อให้ใช้งานง่ายแต่มีประสิทธิภาพ ดังนั้นจึงมีการใช้การส่งข้อมูลแบบเท็กซ์สตรีมแทน

5.1 ความหมายของ HTTP

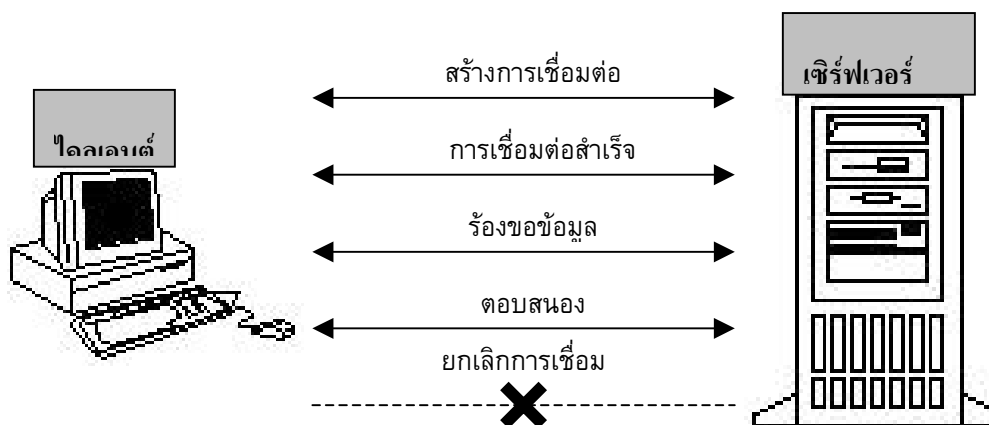
HTTP มาจากคำว่า Hypertext Transfer Protocol ซึ่งเป็นโพรโทคอลที่ใช้ในการส่งข้อมูลต่างๆ ในโลกเวิลด์ไวด์เว็บ ข้อมูลต่างๆ เหล่านี้โดยทั่วไปมักจะถูกเรียกว่า *Resource* โดย Resource เหล่านี้อาจเป็นไฟล์ เช่น ไฟล์ HTML, ไฟล์รูปภาพ หรือคำสั่งต่างๆ (Query String)

HTTP เป็นโพรโทคอลที่อยู่ในส่วนของเลเยอร์แอปพลิเคชัน ในโพรโทคอลสแต็กโดยข้อมูลต่างๆ จากเลเยอร์นี้จะถูกส่งผ่านไปยังเลเยอร์อื่นๆ ที่ต่ำกว่าซึ่งส่วนหนึ่งในนั้นก็คือโพรโทคอล TCP/IP นั่นเอง

HTTP เป็นเน็ตเวิร์คโพรโทคอลที่ใช้หลักการของโมเดลไคลเอนต์เซิร์ฟเวอร์ ในการติดต่อสื่อสารซึ่งมีหลักการทำงานอย่างคร่าวๆ ดังต่อไปนี้

- HTTP ไคลเอนต์ จะทำการสร้างคอนเนกชันไปยัง HTTP เซิร์ฟเวอร์ซึ่งโดยทั่วไปจะผ่านทางซ็อกเก็ตของ TCP/IP
- หลังจากนั้น HTTP ไคลเอนต์ จะทำการส่งคำสั่งการร้องขอ ซึ่งอยู่ในรูปของข้อความไปให้ HTTP เซิร์ฟเวอร์เพื่อขอ Resource ที่ต้องการ
- HTTP เซิร์ฟเวอร์จะทำการตีความคำสั่งที่ได้รับและส่งผลตอบแทนกลับ ซึ่งเป็น Resource ที่ HTTP ไคลเอนต์ ต้องการกลับมา
- หลังจากที่มีการส่งการตอบรับเสร็จสิ้น HTTP เซิร์ฟเวอร์จะทำการปิดคอนเนกชันที่มาจาก HTTP ไคลเอนต์
- ในกรณีที่ HTTP ไคลเอนต์ต้องการ Resource อื่นๆ HTTP ไคลเอนต์ จะต้องทำการสร้างคอนเนกชันใหม่และส่งคำสั่งไปหา HTTP เซิร์ฟเวอร์อีกครั้ง

จากหลักการข้างต้นจะเห็นว่าการติดต่อสื่อสารระหว่างไคลเอนต์และเซิร์ฟเวอร์จะเป็นลักษณะครั้งต่อครั้ง ในทางเครือข่ายเรียกว่าการสื่อสารแบบนี้ว่า Stateless Protocol



รูปที่ 5-1 การติดต่อระหว่างไคลเอนต์กับเซิร์ฟเวอร์

ในการทำงานเว็บเบราว์เซอร์ก็คือ HTTP ไคลเอนต์ นั่นเอง เหตุผลคือเว็บเบราว์เซอร์ถูกใช้เป็นตัวส่งการร้องขอไปที่เว็บเซิร์ฟเวอร์ เพื่อที่จะรับ Resource ที่ต้องการกลับมาโดยใช้โปรโตคอล HTTP โดยทั่วไป Resource จะกระจายอยู่ตามโหนดของเครือข่ายต่างๆ ทั่วโลก สิ่งหนึ่งที่ขาดไม่ได้ในการอ้างถึง Resource เหล่านี้คือ URL (Universal Resource Locator) URL คือตัวที่ใช้ชี้ถึงแหล่งที่อยู่ของ Resource ว่าอยู่ที่ไหน URL สามารถใช้เพื่ออ้างถึง Resource อะไรก็ได้ โดยใช้โปรโตคอลอะไรก็ได้ ไม่ใช่สำหรับ HTTP อย่างเดียวกตัวอย่างเช่น

<http://161.246.5.113/index.html>

เป็น URL ของโปรโตคอล HTTP เพื่อใช้โหลดไฟล์ html

<http://161.246.5.113:80/coffee.jpg>

เป็น URL ของโปรโตคอล HTTP เพื่อใช้โหลดไฟล์รูปภาพ

จากตัวอย่างข้างต้น จะเห็นว่าในหนึ่ง URL จะประกอบไปด้วยชื่อของโปรโตคอล, ชื่อของเซิร์ฟเวอร์, พอร์ตที่ใช้และ Resource ที่ต้องการ

5.2 โครงสร้าง HTTP (HTTP Structure)

จากหลักการทำงานของ HTTP จะเห็นได้ว่าการติดต่อสื่อสารระหว่างไคลเอนต์กับเซิร์ฟเวอร์แล้ว ตัวเมสเสจ(message) จะเป็นตัวกลางที่ใช้ในการติดต่อสื่อสารเสมอ และเพื่อให้ง่ายต่อการใช้งานรูปแบบของเมสเสจที่ใช้ในการร้องขอหรือการตอบรับจึงมีลักษณะคล้ายๆกัน โดยใช้เท็กซ์เป็นหลัก ซึ่งโครงสร้างของเมสเสจจะประกอบด้วย

1. บรรทัดเริ่มต้น (initial line)
2. เฮดเดอร์ (header)
3. บรรทัดว่าง (a blank line) ซึ่งก็คือ CRLF หรือการเว้นหนึ่งบรรทัด (วิธีหนึ่งที่ทำให้คือการกด Enter นั่นเอง)

4. เมสเชงบอดี ซึ่งอาจจะใช้บรรจุไฟล์, คำสั่ง (Query String) หรืออาจจะเป็นเอาต์พุตที่มาจากเซิร์ฟเวอร์โดยส่วนนี้จะมีหรือไม่มีก็ได้ขึ้นอยู่กับจุดประสงค์ของการใช้งาน

CRLF = Carriage return (\r) และ Line Feed (\n) ซึ่งก็คือ \u00A และ \u00D ใน ACSII นั่นเอง
รูปแบบโดยทั่วไปของเมสเชงจะเป็น

```
<initial line>
เซคเตอร์1: value1
เซคเตอร์2: value2
เซคเตอร์3: value3
[blank line]
<เมสเชง body, optional.....>
.....>
```

ตัวอย่างง่ายๆของเมสเชงที่ไคลเอนต์ใช้อาจจะเป็น

```
GET /images/cat.jpg      ← บรรทัดเริ่มต้น
From: soup@jarticles.com ← เซคเตอร์
User-Agent: Mozilla/4.72 ← เซคเตอร์
[blank line]
หรือ
POST /servlet/searchEngine HTTP/1.0 ← บรรทัดเริ่มต้น
From: webmaster@jarticles.com ← เซคเตอร์
User-Agent: Mozilla/4.72 ← เซคเตอร์
[blank line]
keyword=java&topic=jsp ← เมสเชงบอดี
```

ตัวอย่างง่ายๆของเมสเชงที่เซิร์ฟเวอร์ใช้อาจจะเป็น

```
HTTP 1.0 200 OK          ← บรรทัดเริ่มต้น
Date: Sunday, 23-July-00 04:01:12 GMT ← เซคเตอร์
Server: Apache/1.3.12(Unix) (Red Hat/Linux) PHP/3.0.15 mod_perl/1.21 ← เซคเตอร์
MIME-version: 1.0       ← เซคเตอร์
Content-type: text/html  ← เซคเตอร์
Content-length: 115      ← เซคเตอร์
[blank line]
<HTML><HEAD><TITLE>HTTP Tutorial</TITLE></HEAD> ← เมสเชงบอดี
```

<BODY>This is a tutorial, but please visit me again . . . </BODY> ← เมสเซจบอดี(cont.)
 </HTML> ← เมสเซจบอดี(cont.)

5.2.1 บรรทัดเริ่มต้นในการร้องขอ (Initial Request Line)

บรรทัดเริ่มต้นของการร้องขอ(จากไคลเอนต์)จะแตกต่างจากบรรทัดเริ่มต้นของการตอบรับเล็กน้อย โดยบรรทัดเริ่มต้นของการร้องขอจะมี 3 ส่วนคือ

- ชื่อของเมธอด เช่น GET, POST, HEAD, TRACE
- โคลอเลพาธของ resource ที่ไคลเอนต์ต้องการ
- เวอร์ชันของ HTTP/x.x ที่ HTTP ไคลเอนต์ใช้

สามส่วนนี้จะประกอบกันเป็นบรรทัดเริ่มต้น โดยแต่ละส่วนจะถูกแยกออกจากกันโดยใช้ช่องว่าง (space) ดังตัวอย่างข้างล่าง

GET /path/to/file/index.html Http/1.0

ตัวอย่างนี้ใช้เมธอดที่ชื่อ GET เพื่อขอ resource (ในที่นี้ก็คือไฟล์) ชื่อ /path/to/file/index.html โดยใช้โปรโตคอล HTTP/1.0 ในการสื่อสาร

การใช้โคลอเลพาธ (Local path)

ก่อนที่ HTTP ไคลเอนต์จะส่งการร้องขอไปที่ HTTP เซิร์ฟเวอร์ HTTP ไคลเอนต์จะสร้างการเชื่อมต่อขึ้นมาก่อนหนึ่งก่อนโดยใช้ชื่อที่เกิด ในการสร้างชื่อเกิดจะต้องทำการกำหนดชื่อของโฮสต์ที่จะติดต่อ ยกตัวอย่างเช่น ถ้า URL เป็น <http://161.246.5.117/path/to/file/index.html> ชื่อของโฮสต์ก็จะเป็น 161.246.5.117 ดังนั้นการกำหนดเพียงโคลอเลพาธเพื่อบ่งบอกถึง resource ที่ต้องการในส่วนของการร้องขอเริ่มต้นในการร้องขอก็คือ /path/to/file/index.html ก็เพียงพอแล้ว

API ที่ใช้ในแอปพลิเคชันทางฝั่งเซิร์ฟเวอร์ มักอ้างถึงส่วนที่เป็นโคลอเลพาธนี้ว่า “Request URI (ตัว I มาจาก Identifier)”

ข้อระวัง

- ชื่อของเมธอดจะต้องใช้ตัวใหญ่เสมอ
- เวอร์ชันของ HTTP จะอยู่ในรูป HTTP/x.x และจะต้องเป็นตัวใหญ่เสมอ

5.2.2 บรรทัดเริ่มต้นในการตอบสนอง (Initial Response Line)

โดยทั่วไปบรรทัดเริ่มต้นในการตอบสนอง(จากเซิร์ฟเวอร์)มักจะเรียกว่า Status line ซึ่งจะประกอบไปด้วย 3 ส่วนย่อยคือ

- 1 เวอร์ชันของ HTTP/x.x ที่เซิร์ฟเวอร์ใช้สำหรับส่งเมสเซจ
 - 2 Response status code ซึ่งเป็นตัวบอกว่าผลของการร้องขอที่ไคลเอนต์ส่งมาเป็นอย่างไร
 - 3 Reason phase เป็นตัวอธิบายความหมายของ Response status code อีกทีหนึ่ง
- ยกตัวอย่างเช่น

HTTP/1.0 200 OK

หรือ

HTTP/1.0 404 Not Found

ข้อสังเกต

- เวอร์ชันของ HTTP จะต้องอยู่ในรูป HTTP/x.x และจะต้องเป็นตัวใหญ่เสมอ
- โค้ดสถานะการตอบสนอง (response status code) เป็น โค้ดที่ส่งมาให้คอมพิวเตอร์ทางฝั่งไคลเอนต์อ่านซึ่งจะมี reason phase เป็นตัวอธิบายให้ผู้ใช้งาน (Human Being) อ่านอีกทีหนึ่ง
- โค้ดสถานะการตอบสนองจะอยู่ในลักษณะของเลข 3 หลัก โดยหลักแรกจะบอกถึงความหมายโดยทั่วไปของ response
 - 1xx เป็น code ที่ใช้บอกถึงเหตุการณ์ต่างๆที่เกิดขึ้นในการสื่อสารระหว่างไคลเอนต์และเซิร์ฟเวอร์
 - 2xx เป็น code ที่บ่งบอกว่า request ที่ส่งจากไคลเอนต์ถูกทำให้สมบูรณ์แล้ว
 - 3xx เป็น code ที่บอกให้ไคลเอนต์ทำการ redirect ไปที่ url ที่อื่นแทน
 - 4xx เป็น code ที่บ่งบอกถึงความผิดพลาดที่เกิดขึ้นในส่วนของไคลเอนต์
 - 5xx เป็น code ที่บ่งบอกถึงความผิดพลาดที่เกิดขึ้นในส่วน of เซิร์ฟเวอร์

โค้ดแสดงสถานะ (Status code) ที่พบเห็นโดยทั่วไป คือ

- 100 Continue หมายถึง เซิร์ฟเวอร์ได้รับ request บางส่วนจากไคลเอนต์แล้ว บอกให้ไคลเอนต์ส่งที่เหลือมาด้วย
- 200 OK หมายถึง คำร้องขอที่มาจากไคลเอนต์ถูกต้องและ response ที่ไคลเอนต์ต้องการอยู่ในเมสเซจบอดี้ของ response นี้แล้ว
- 301 Moved Permanently หมายถึง resource ที่ไคลเอนต์ต้องการ ได้ถูกย้ายไปที่อื่นแล้ว
- 302 Moved Temporarily หมายถึง resource ที่ไคลเอนต์ต้องการ ได้ถูกย้ายไปอยู่ที่อื่นชั่วคราว
- 400 Bad Request หมายถึง เซิร์ฟเวอร์ไม่สามารถเข้าใจการร้องขอที่ไคลเอนต์ส่งมา
- 403 Forbidden หมายถึง เซิร์ฟเวอร์เข้าใจการร้องขอที่ไคลเอนต์ส่งมาแต่ไม่สามารถที่จะส่ง resource ที่ไคลเอนต์ต้องการกลับไปได้
- 404 Not Found หมายถึง เซิร์ฟเวอร์ไม่มี resource ที่ร้องขอมา
- 500 Server Error หมายถึงข้อผิดพลาดที่เกิดขึ้นในส่วน of เซิร์ฟเวอร์ (โดยทั่วไปข้อผิดพลาดนี้จะเกิดขึ้นในกรณีที่มีการร้องขอส่งมาเรียกแอพพลิเคชันฝั่งเซิร์ฟเวอร์ที่รันบนเซิร์ฟเวอร์แต่เกิดข้อผิดพลาดขึ้นระหว่างการประมวลผล ซึ่งโดยทั่วไปจะเกิดขึ้นเนื่องจากโปรแกรมมีบั๊ก(bug))

5.2.3 เฮดเดอร์ (Header)

เฮดเดอร์จะเป็นส่วนที่บอกถึงรายละเอียดของการร้องขอ (ที่กำลังส่งไปให้เซิร์ฟเวอร์) หรือการตอบรับ (ที่กำลังส่งกลับมายังไคลเอนต์) ยกตัวอย่างเช่น

- ไคลเอนต์จะส่งเฮดเดอร์ที่บอกถึงชนิดและขนาดของข้อมูลที่อยู่ข้างในเมสเซจบอดี ในกรณีที่ไคลเอนต์ต้องการ upload ไฟล์ไปยังเซิร์ฟเวอร์
- เซิร์ฟเวอร์อาจจะส่งเฮดเดอร์ที่เกี่ยวกับชนิดและขนาดของ resource ที่ไคลเอนต์กำลังจะได้รับ โดยใช้ Content-Type และ Content-Length
- เฮดเดอร์จะมีรูปแบบเป็นเท็กซ์ โดยในหนึ่งบรรทัดจะถูกใช้สำหรับหนึ่งเฮดเดอร์ซึ่งจะอยู่ในรูปของ “เฮดเดอร์-Name: value” และจบด้วย CRLF ยกตัวอย่างเช่น

From: admin@ce.kmitl.ac.th

User-Agent: Mozilla/4.72

Content-Type: text/html

Content-Length: 250

ข้อสังเกต

- ชื่อของเฮดเดอร์จะใช้ตัวใหญ่หรือตัวเล็กก็ได้
- สามารถเว้นช่องว่างหรือ tab ระหว่าง “:” ของชื่อเฮดเดอร์และค่าของมันกว้างเท่าไรก็ได้
- ใน HTTP1.0 จะมีเฮดเดอร์กำหนดไว้ 16 แบบ แต่ของ HTTP1.1 จะมีถึง 46 แบบ

เฮดเดอร์ที่มักจะพบเห็นทั่วไปในส่วนของไคลเอนต์

From: เป็นเฮดเดอร์ที่ใช้เก็บอีเมลแอดเดรสของผู้ที่กำลังส่งการร้องขอ ยกตัวอย่างเช่น

From: admin@ce.kmitl.ac.th

User-Agent: เป็นเฮดเดอร์ที่ใช้บ่งบอกถึงชื่อของโปรแกรมที่ใช้เป็น HTTP ไคลเอนต์ ยกตัวอย่างเช่น User-Agent: Mozilla/4.72

เฮดเดอร์ที่มักพบเห็นทั่วไปในส่วนของเซิร์ฟเวอร์

Server: เป็นเฮดเดอร์ที่จะคล้ายกับ User-Agent แต่เป็นตัวบ่งบอกถึงชื่อโปรแกรมที่ใช้เป็น HTTP เซิร์ฟเวอร์ ยกตัวอย่างเช่น Server: Apache/1.93

Last-Modified: เป็นเฮดเดอร์ที่ใช้บ่งบอกถึงเวลาครั้งสุดท้ายที่ resource ที่ไคลเอนต์ต้องการถูกเปลี่ยนแปลง (modify/update) โดยเวลาที่ใช้จะเทียบจาก GMT (Green wich Mean Time) ยกตัวอย่างเช่น เวลาของเมืองไทยเป็น GMT+7.00 ถ้าตอนนี้เวลาเมืองไทยคือ Sun, 23 July 2000 20:39:56 เวลาที่เป็น GMT ก็คือ Last-Modified: Sun, 23 July 2000 13:39:56 GMT

5.2.4 เมสเซจบอดี (message body)

เมื่อใดก็ตามที่ไคลเอนต์หรือเซิร์ฟเวอร์ต้องการส่งข้อมูลไปกับเมสเซจ ส่วนที่เป็นเมสเซจบอดีจะเป็นส่วนที่ใช้เก็บข้อมูลดังกล่าว ในกรณีของไคลเอนต์ตัวเมสเซจบอดีอาจใช้สำหรับบรรจุไฟล์ที่ต้องการอัปโหลดหรือใช้เก็บข้อมูลที่มาจากอีลิเมนต์ต่างๆ ของฟอร์ม HTML แล้วส่งไปยังเซิร์ฟเวอร์ก็ได้ ในกรณีของเซิร์ฟเวอร์ตัวเมสเซจบอดีจะเป็นส่วนที่ใช้เก็บ resource ที่ไคลเอนต์ร้องขอหรืออาจใช้เก็บคำอธิบายต่างๆ ในกรณีที่มีข้อผิดพลาดเกิดขึ้นก็ได้ ถ้าเมสเซจมีส่วนของเมสเซจบอดี ไคลเอนต์หรือเซิร์ฟเวอร์มักจะเพิ่มเฮดเดอร์ที่ช่วยบ่งบอกถึงรายละเอียดของเมสเซจบอดีดังกล่าวด้วย ยกตัวอย่างเช่นถ้าเซิร์ฟเวอร์ต้องการส่ง resource ที่เป็นไฟล์รูปภาพมาให้ไคลเอนต์ เฮดเดอร์ที่ถูกเพิ่มมาจะเป็น Context-Type: image/gif เป็นเฮดเดอร์ที่บอกถึงMIME type ของข้อมูลที่อยู่ในเมสเซจบอดี Context-Length: 1026 เป็นเฮดเดอร์ที่บ่งบอกถึงขนาดของเมสเซจบอดีในหน่วยไบต์

ตัวอย่างของการส่งเมสเซจระหว่างไคลเอนต์ (Web Browser) และเซิร์ฟเวอร์ (Web Server) เช่น

<http://161.246.5.117/tutorials/basic/helloworld.html>

ขั้นแรกไคลเอนต์จะทำการเรียกใช้ช็อกเก็ตเพื่อเชื่อมต่อไปยังเซิร์ฟเวอร์ชื่อ 161.246.5.117 ซึ่งโดยปกติจะติดต่อไปที่พอร์ต 80 ซึ่งเป็นพอร์ต Default ของโพรโทคอล HTTP (ในกรณีที่ต้องการเชื่อมต่อไปยังพอร์ตอื่นก็สามารถทำได้โดยการเพิ่มส่วนของเลขพอร์ตที่ไคลเอนต์ต้องการเชื่อมต่อเข้าไปหลังจากส่วนของโฮสต์เนม เช่น <http://161.246.5.117:8080/tutorials/advance/helloworld.html> ทำให้ไคลเอนต์เชื่อมต่อไปยังพอร์ต 8080 แทน)

หลังจากนั้นไคลเอนต์จะทำการส่งเมสเซจร้องขอ

GET /tutorials/basic/helloworld.html HTTP/1.0

From: soup@jarticles.com

User-Agent: Mozilla/4.72

[blank line]

หลังจากที่เซิร์ฟเวอร์ได้รับเมสเซจข้างบนแล้ว เซิร์ฟเวอร์ส่งการตอบรับพร้อมทั้ง resource ที่ไคลเอนต์ต้องการกลับมานี้

HTTP/1.0 200 OK

Date: Sunday, 23-July-00 04:01:12 GMT

Content-Type: text/html

Content-Length: 1556

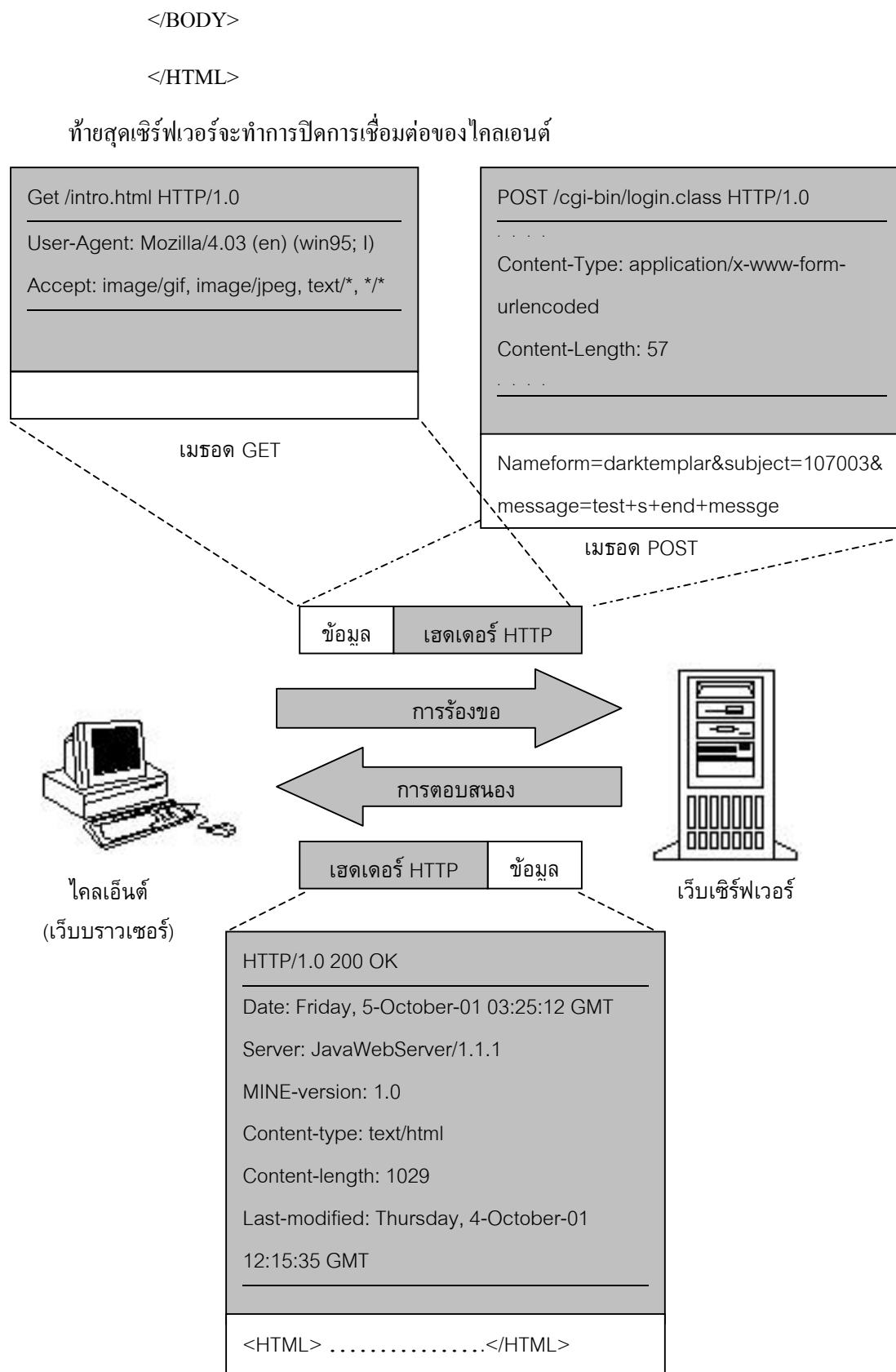
[blank line]

<HTML><HEAD><TITLE>HTTP Tutorial</TITAL></HEAD>

<BODY>This is HelloWorld tutorial, but please read it for fun

(more file contents)

...



รูปที่ 5-2 ลักษณะข้อมูลที่รับส่งระหว่างไคลเอนต์และเซิร์ฟเวอร์

5.3 เมธอด (Method)

5.3.1 เมธอด GET

ตัวอย่างเช่น <http://www.google.com/search.cgi?keyword=computer> เป็นการค้นหาที่ www.google.com โดยใช้คีย์เวิร์ด = “computer”

ถ้าพิจารณาส่วนที่อยู่ต่อจากส่วนของโฮสต์เนม (www.google.com) แล้วจะเห็นว่า URL ที่ถูกร้องขอ (/search?keyword=computer) ที่เห็นไม่ได้เป็นไฟล์ html ไฟล์อย่างทั่วๆ ไปแต่กลับกลายเป็นชื่อของโปรแกรม (search.cgi), เครื่องหมายคำถาม (?) และคีย์เวิร์ดที่ใช้ค้นหาแทน อย่างที่กล่าวมาแล้วในส่วนต้นของบทความแล้วว่า Resource จริงๆ แล้วคือตัวที่ใช้บ่งชี้ถึงข้อมูลที่อยู่ในที่ server resource อาจเป็นไฟล์หรือเป็น Query String ที่ใช้ส่งไปเรียกโปรแกรมที่อยู่ในที่เซิร์ฟเวอร์ก็ได้ ซึ่งในกรณีตัวอย่างของ Search Engine ข้างต้น ตัว Resource ก็คือโปรแกรมที่ชื่อ search.cgi นั่นเอง โดยทั่วไปการตอบรับที่ได้กลับมาจากเซิร์ฟเวอร์อาจอยู่ในรูปของไฟล์ถ้า Resource ที่ร้องขอเป็นไฟล์

โดยทั่วไปเว็บเบราว์เซอร์(ไคลเอนต์) จะใช้เมธอด GET ในการส่งเมสเสจการร้องขอไปที่เซิร์ฟเวอร์ในขณะที่ Resource ที่ต้องการเป็นไฟล์ อย่างไรก็ตาม GET ยังสามารถใช้ในการส่ง Query String สั้นๆ ไปยังโปรแกรมที่รันอยู่เซิร์ฟเวอร์ได้อีกด้วย ตัวอย่างเช่น ถ้าต้องการส่งข้อมูลชื่อคีย์เวิร์ดโดยมีค่าเท่ากับ computer ไปที่เซิร์ฟเวอร์ชื่อ www.google.com โดยจะส่งไปที่ Resource ที่เป็น cgi โปรแกรมชื่อ search.cgi วิธีทำคือ

- กำหนด resource ลงไปส่วนของ request URI ซึ่งจะอยู่หลังจากส่วนของโฮสต์เนมซึ่งจะได้ <http://www.google.com/search.cgi>
- ใส่ตัวเครื่องหมายคำถามเพื่อบอกเซิร์ฟเวอร์ว่าส่วนที่เหลือถัดไปจะเป็นส่วนของข้อมูลซึ่งจะได้ <http://www.google.com/search.cgi?>
- ทำการเข้ารหัสโดยวิธีการที่เรียกว่า *URI-encoding* ไปที่ชื่อและค่าของตัวแปรต่างๆ ซึ่งในกรณีของตัวแปรก็คือคีย์เวิร์ดซึ่งจะได้ <http://www.google.com/search.cgi?keyword=name>

ถ้าหากพิมพ์ URL ข้างบนเข้าไปในเว็บเบราว์เซอร์แล้วกดปุ่ม enter ตัวเว็บเบราว์เซอร์จะทำการตีความ URL ดังกล่าวซึ่งผลที่ได้คือการใช้ GET ส่งเมสเสจขอไปที่เซิร์ฟเวอร์ที่ชื่อ www.google.com โดยจะแนบคีย์เวิร์ด “computer” ไปกับการร้องขอซึ่งบรรทัดเริ่มต้นของเมสเสจการร้องขอจะเป็นอะไรคล้ายๆ ข้างล่างนี้

GET/search.cgi?keyword=jarticles HTTP/1.0

From: soup@jarticles.com

User-Agent: Mozilla/4.72

[blank line]

หลังจากนั้นจะได้ลิงค์ต่างๆ ที่เกี่ยวกับ computer กลับมา สังเกตว่า URL จริงๆ ที่ใช้กับ www.google.com จะต่างจาก URL ข้างล่างเล็กน้อย วิธีการส่ง query string ไปกับ URL เหมาะสำหรับ query string ที่ไม่ยาว

มากนัก โดยทั่วไปความของ URL มักจะจำกัดอยู่ที่ 80 ตัวอักษร ซึ่งโดยทั่วไปแล้วตัวอักษรที่ถัดจากนั้นจะถูกตัดออกไปโดยอัตโนมัติ

การเข้ารหัส URL (URL -Encoding)

ในการส่งข้อมูลไปที่ เซิร์ฟเวอร์โดยวิธีการ GET หรือ POST นั้นชื่อและค่าของตัวแปรต่างๆ ซึ่งอยู่หลังจากเครื่องหมายคำถามในกรณีของ GET หรืออยู่ในส่วนของ เมสเซจบอดีในกรณีของ POST จะต้องถูกผ่านการเข้ารหัสโดยวิธีการที่เรียกว่า *URI-Encoding* เสียก่อน โดยทั่วไปตัว URL เองจะมีตัวอักษรบางตัวที่ใช้สำหรับความหมายพิเศษ เช่น :, /, ~, & และ ? ซึ่งในบางครั้งชื่อและค่าของตัวแปรต่างๆ ที่ถูกส่งไปที่ เซิร์ฟเวอร์อาจจะมีตัวอักษรเหล่านี้ปะปนอยู่ด้วย เพื่อหลีกเลี่ยงความสับสนในการตีความของเซิร์ฟเวอร์ ตัวอักษรต่างๆ ที่เป็นที่ใช้ใน URL จะต้องถูกการเข้ารหัสเสียก่อน โดยมีหลักการดังต่อไปนี้ให้ทำการเปลี่ยน unsafe characters ต่างๆ ที่อยู่ในชื่อและค่าของตัวแปรให้กลายเป็น “%xx” โดย “xx” คือค่าของแอสกีของตัวอักษรนั้นๆ ในแบบเลขฐาน 16 ซึ่ง ตัวอักษร unsafe เหล่านี้รวมไปถึง =, &, %, + และตัวอักษร ต่างๆ ที่ไม่สามารถพิมพ์ได้แม้กระทั่ง character ที่ต้องการจะเข้ารหัส

- ให้ทำการเปลี่ยนช่องว่าง(space) ทั้งหมดให้กลายเป็นเครื่องหมายบวก (+)
- จับคู่ชื่อและค่าของตัวแปรต่างๆ ด้วยเครื่องหมายเท่ากับ (=)
- ถ้าชื่อและค่าของตัวแปรมากกว่าหนึ่งตัว ให้นำชื่อและค่าของตัวแปรแต่ละคู่มาเชื่อมติดกันด้วยเครื่องหมาย &
- นำชื่อและค่าของตัวแปรที่เชื่อมติดกันทั้งหมดไปใส่ที่ URL โดยมีเครื่องหมาย ? อยู่ข้างหน้า(กรณีของ GET) หรือนำไปใส่ที่เมสเซจบอดี(กรณีของ POST) ยกตัวอย่างเช่น ถ้ามีชื่อและค่าดังนี้

(name, value) = ("keyword", "jarticles")

(name, value) = ("name", "Soup & friends")

หลังจากทำการเข้ารหัสจะได้ String ออกมาเป็น keyword=jarticles&name=Soup+%26+friends โดยStringนี้มีความยาวเท่ากับ 39 โดยที่ & = %26 ในเลขฐานสิบหก

5.3.2 เมธอด POST

บางครั้งหากไม่ต้องการส่งข้อมูลไปยังเซิร์ฟเวอร์ด้วยการติดข้อมูลเหล่านั้นไปกับ URL ด้วยเหตุผลที่ว่าข้อมูลเหล่านั้นอาจเกี่ยวข้องกับข้อมูลส่วนตัว ยกตัวอย่างเช่น login และ password ที่ใช้เป็นตัวผ่านเข้าไปในเว็บไซด์ต่างๆ ซึ่งในกรณีที่ใช้ GET, URL, หลังจากที่ทำ login ไปแล้ว URL ที่อยู่ตรง Address Bar อาจจะออกมาเป็น http://www.myserver.com/login.cgi?login=me&password=youandme ซึ่งอาจคนเห็นและนำข้อมูลไปใช้อย่างผิดๆ ได้อีกกรณีหนึ่งคือกรณีของข้อมูลที่ติดไปกับส่วนของ URL ที่ถูกร้องขอที่ทำให้ URL มีความยาวมากกว่าความยาวของ URL ที่เซิร์ฟเวอร์กำหนดไว้ ผลกระทบที่อาจเกิดขึ้นคือข้อมูลบางส่วนอาจหายไปเนื่องจากตัดทอนโดยเซิร์ฟเวอร์

วิธีการที่สามารถใช้เพื่อปิดบังข้อมูล หรือเพื่อส่งข้อมูลที่มีความยาวมากๆ ก็คือการใช้เมธอด POST โดยที่ POST ต่างกับ GET ตรงที่ POST จะไม่ทำการติดข้อมูลไปกับ URL แต่ POST จะทำการใส่ข้อมูลเข้าไปในส่วนของเมสเสจบอดีของเมสเสจร้องขอแทน ซึ่งในกรณีนี้ URI ที่ถูกร้องขอจะปรากฏเพียงชื่อของโปรแกรมที่ต้องการส่งข้อมูลไปให้เท่านั้น ในการส่งข้อมูลไปในส่วนของเมสเสจบอดี เพื่อป้องกันการตีความโดยเซิร์ฟเวอร์ทางเมธอด POST จึงมีการบังคับให้ใส่เฮดเดอร์พิเศษเพื่อบรรยายรายละเอียดต่างๆ ของข้อมูลที่อยู่ในเมสเสจบอดีนั่นอีกด้วย ซึ่งโดยทั่วไปเฮดเดอร์ที่มักถูกเพิ่มเข้าไปก็คือ Content-Type และ Content-Length การใช้ POST ที่มักพบเห็นโดยทั่วไปก็คือการใช้ POST ในการส่งข้อมูลที่มาจากรูปแบบ HTML ไลอเนนจ์จะทำการเซต Content-Type เป็น application/x-www-form-urlencoded และ Content-Length จะเท่ากับผลรวมของความยาวของชื่อและค่าของอิลิเมนต์ต่างๆ ที่อยู่ในฟอร์มของ HTML รวมกันซึ่งตัวอย่างก็คืออิลิเมนต์ฟอร์มที่ชื่อ username และ password นอกจากนี้ POST ยังสามารถใช้ในการส่งข้อมูลแบบอื่นๆ ได้อีกด้วย โดยรูปแบบขึ้นอยู่กับวิธีการส่งที่ทางเว็บเบราว์เซอร์ และเว็บเซิร์ฟเวอร์ได้ทำการตกลงกันไว้ ยกตัวอย่างเช่น ในกรณีของไฟล์อัปโหลดตัว Content-Type ก็จะถูกเซตเป็น Multipart/form-data แทน